Assembly Language

- DosBox https://www.dosbox.com/
- NASM https://www.nasm.us/
- Insight https://www.bttr-software.de/products/insight/
- Complete 8086 instruction set https://www.eng.auburn.edu/~sylee/ee2220/8086_instruction_set.html

DosBox

- **DOSBox** is an open-source emulator that allows you to run older software, particularly programs and games designed for **MS-DOS** (Microsoft Disk Operating System), on modern hardware and operating systems.
- MS-DOS was widely used in the 1980s and early 1990s but has since become obsolete. Many older programs and games were written specifically for this environment, and DOSBox provides a virtual machine to emulate this system so these applications can still run today.

NASM

- **NASM (Netwide Assembler)** is an open-source assembler for the **x86** and **x86-64** architectures, commonly used to write low-level programs like operating system kernels, bootloaders, and other system-level software. It translates assembly language code into machine code that the processor can execute.
- **Cross-Platform**: NASM runs on various platforms, allowing developers to write assembly code for a variety of operating systems and architectures.

Insight - real-mode DOS debugger

- Insight is a very small debugger for analyzing real-mode DOS programs.
- It features an i80486 disassembler, an i8086 assembler, 'Trace into' and 'Step over' functions, simple breakpoint handling, extended code or data navigation, simple color-highlighting, and a nice menu-driven interface comparable to Borland's Turbo Debugger.
- Copy insight.com to working directory

hello.asm

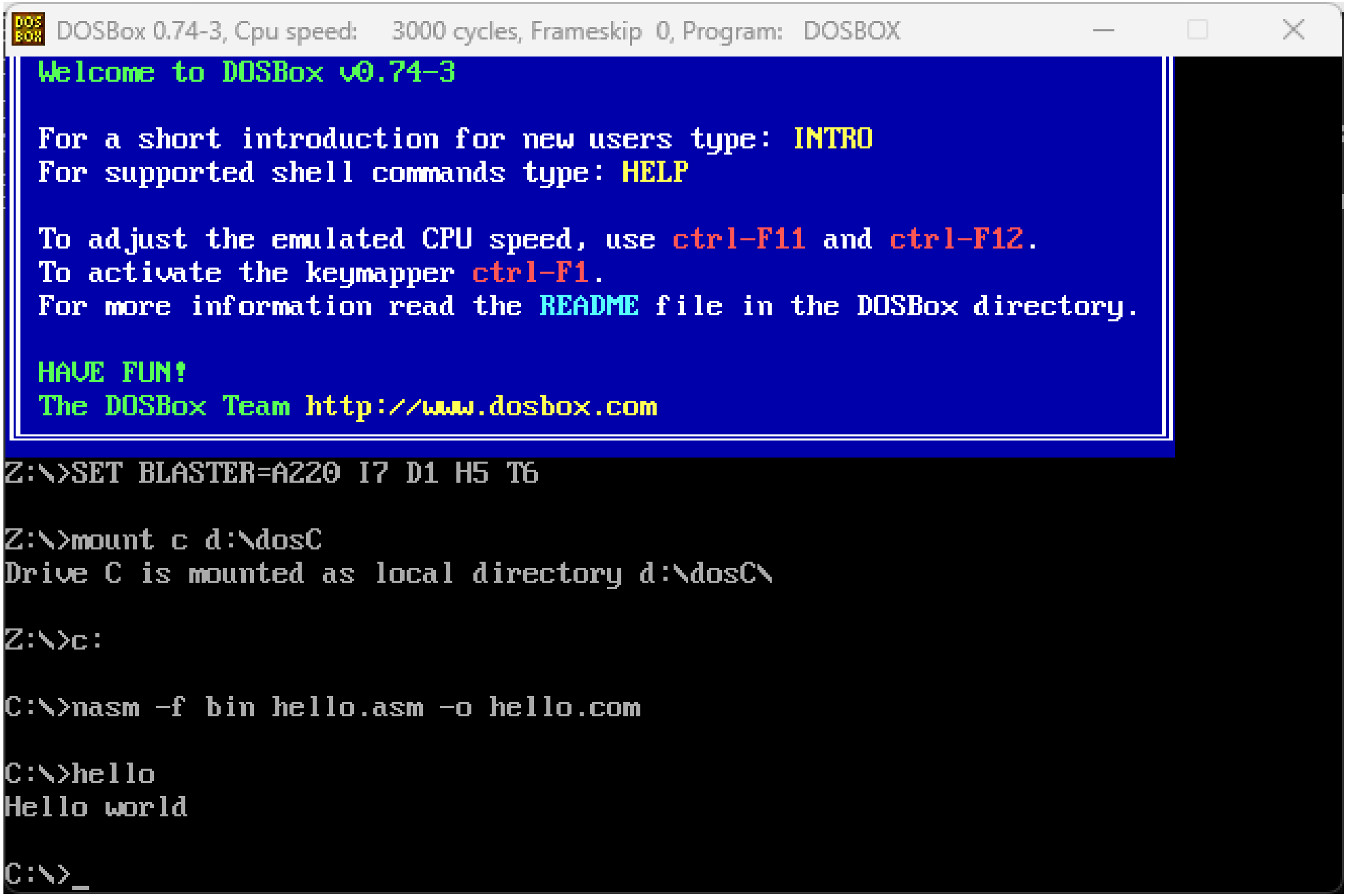
```
; 16-bit COM file example
org 100h

section .text

start:
  ; program code
  mov  dx, msg
  mov  ah, 09h
  int  0x21
  int 20h

section .data
  ; program data
  msg  db 'Hello world$’
```

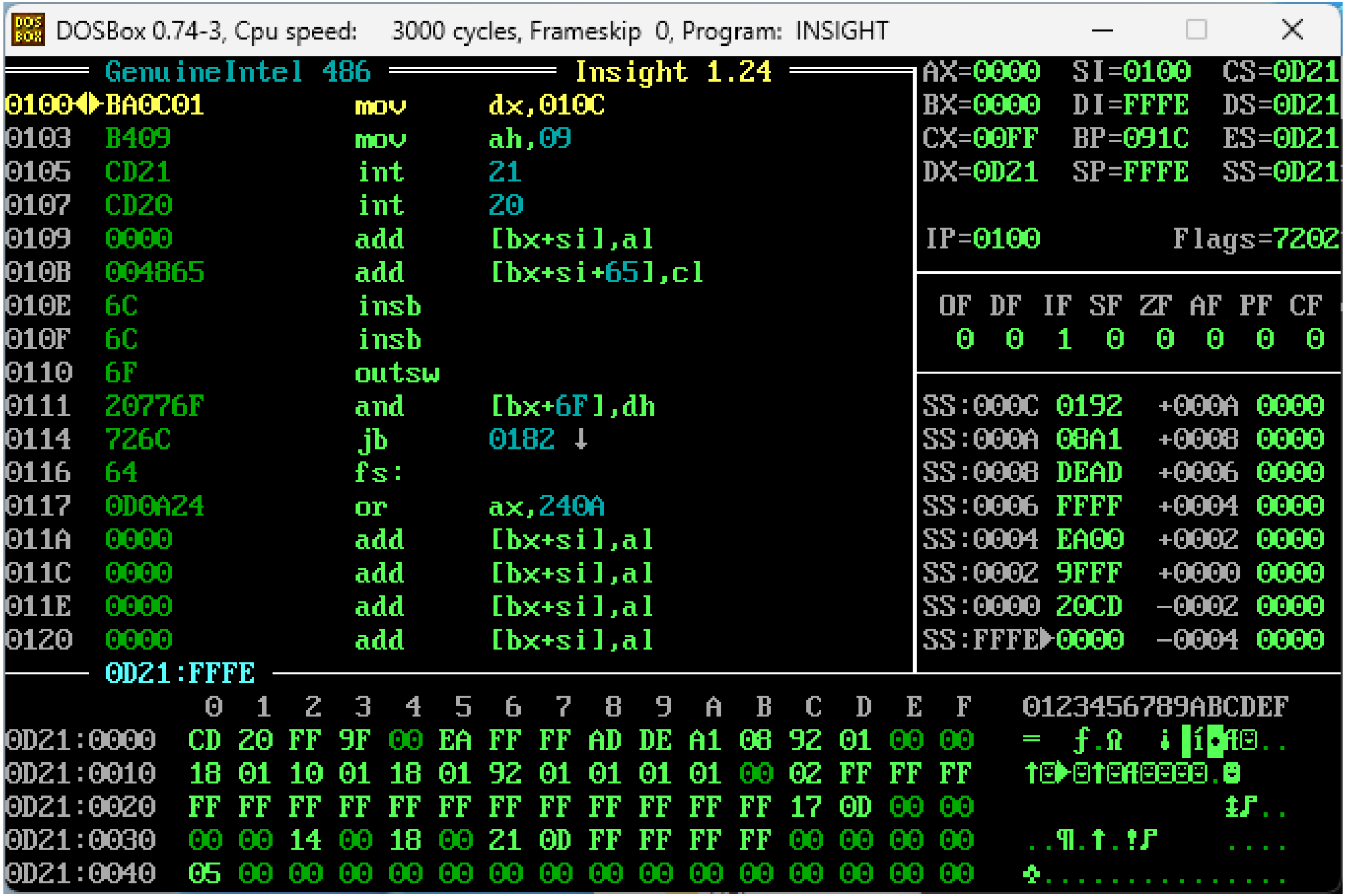
Run your first assembler language program

- Download and install dosbox
- Extract nasm+afd to a folder, for example: d:\dosC
- Open DosBox, mount the directory d:\dosfile to virtual drive C: using the following command:

  mount c d:\dosC
- In DosBox, change to C:
- Compiler hello.asm using command

  nasm -f bin hello.asm -o hello.com
- Execute the program by typing hello and press Enter
- Debug tool: insight hello.com


DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: DOSBOX
Welcome to DOSBox v0.74-3
For a short introduction for new users type: INTRO
For supported shell commands type: HELP
To adjust the emulated CPU speed, use ctrl-F11 and ctrl-F12.
To activate the keymapper ctrl-F1.
For more information read the README file in the DOSBox directory.
HAVE FUN!
The DOSBox Team http://www.dosbox.com
Z:\>SET BLASTER=A220 I7 D1 H5 T6
Z:\>mount c d:\dosC
Drive C is mounted as local directory d:\dosC\
Z:\>c:
C:\>nasm -f bin hello.asm -o hello.com
C:\>hello
Hello world
C:\>_


DOSBox 0.74-3, Cpu speed: 3000 cycles, Frameskip 0, Program: INSIGHT
GenuineIntel 486
Insight 1.24
0100 BA0C01 mov dx,010C
0103 B409 mov ah,09
0105 CD21 int 21
0107 CD20 int 20
0109 0000 add [bx+si],al
010B 004865 add [bx+si+65],cl
010E 6C insb
010F 6C insb
0110 6F outsw
0111 20776F and [bx+6F],dh
0114 726C jb 0182 ↓
0116 64 fs:
0117 0D0A24 or ax,240A
011A 0000 add [bx+si],al
011C 0000 add [bx+si],al
011E 0000 add [bx+si],al
0120 0000 add [bx+si],al
AX=0000 SI=0100 CS=0D21
BX=0000 DI=FFFE DS=0D21
CX=00FF BP=091C ES=0D21
DX=0D21 SP=FFFE SS=0D21
IP=0100 Flags=7202
OF DF IF SF ZF AF PF CF
0 0 1 0 0 0 0 0
SS:000C 0192 +000A 0000
SS:000A 08A1 +0008 0000
SS:0008 DEAD +0006 0000
SS:0006 FFFF +0004 0000
SS:0004 EA00 +0002 0000
SS:0002 9FFF +0000 0000
SS:0000 20CD -0002 0000
SS:FFFE 0000 -0004 0000
0D21:FFFE
0 1 2 3 4 5 6 7 8 9 A B C D E F
0123456789ABCDEF
0D21:0000 CD 20 FF 9F 00 EA FF FF AD DE A1 08 92 01 00 00
0D21:0010 18 01 10 01 18 01 92 01 01 01 01 00 02 FF FF FF
0D21:0020 FF FF FF FF FF FF FF FF FF FF FF FF 17 0D 00 00
0D21:0030 00 00 14 00 18 00 21 0D FF FF FF FF 00 00 00 00
0D21:0040 05 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00

8086 segment:offset Addressing

- The value in any register considered to be a Segment register is multiplied by 16 (or shifted one hexadecimal byte to the left; add an extra 0 to the end of the hex number) and then the value in an Offset register is added to it. So, the Absolute address for any combination of Segment and Offset pairs is found by using the formula:

Absolute Memory Location = (Segment value * 16) + Offset value

segment:offset pair F000:FFFD = FFFFD

https://thestarman.pcministry.com/asm/debug/Segments.html

8086 Registers

- These registers are grouped into 3 categories:
  - Segment registers
  - General registers
  - Control registers

https://www.tutorialspoint.com/microprocessor/microprocessor_8086_overview.htm

Segment Registers

- **Code Segment (CS)**
  - It contains all the instructions to be executed
  - A 16-bit Code Segment Register or CS Register stores the starting address of the code segment
- **Data Segment (DS)**
  - It contains all the variables (data)
  - A 16-bit Data Segment Register or DS Register stores the starting address of the data segment
- **Extra/ Extended Segment (ES)**
  - It is used to store variables (data), incase the data segment is not sufficient to store all variables
  - A 16-bit Extra/ Extended Segment Register or ES Register stores the starting address of the extra segment
- **Stack Segment (SS)**
  - It contains data and return addresses of subroutines
  - A 16-bit Stack Segment Register or SS Register stores the starting address of the stack segment

General Registers

- **Data Registers** Four 16-bit data registers are used for arithmetic, logical and other operations
  - AX(AH,AL) -Accumulator Register
  - BX(BH,BL) -Base Register
  - CX(CH,CL) -Count Register
  - DX(DH,DL) -Data Register
- **Pointer Registers** Two 16-bit pointer registers used in 16-bit architecture are:
  - SP -Stack Pointer It works with SS register (SS:SP)
  - BP -Base Pointer It works with SS register (SS:BP)
- **Index Registers** Two 16-bit index registers are used for indexed addressing and sometimes used in addition and subtraction
  - SI - Source Index It works with DS (DS:SI)
  - DI- Destination Index It works with ES (ES:DI)

Control Registers

- **The 16-bit Instruction Pointer (IP) Register**
  - It is used to store offset address of the next instruction to be executed
  - It works with CS (CS:IP)

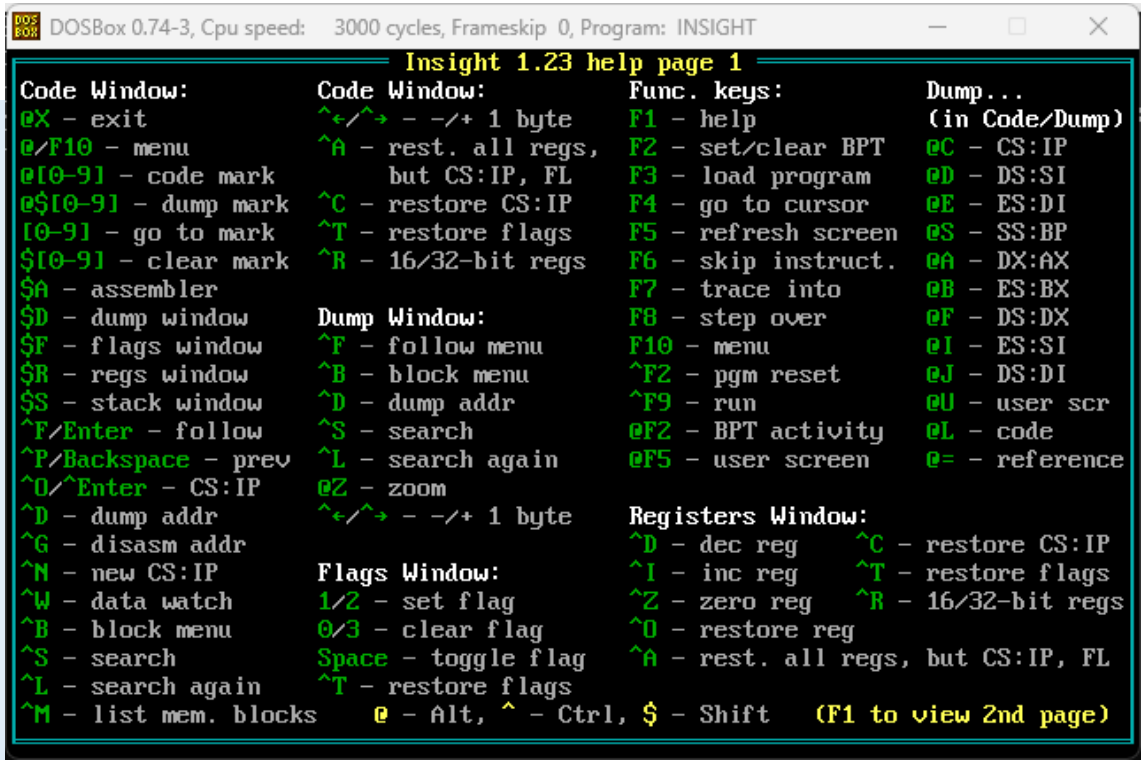
Flags Register It is used to store the status value after arithmetic and logic operations that are tested by conditional instructions to change the flow of program execution

Common flag bits are:

Sign Flag	-SF
Overflow Flag	-OF
Direction Flag	-DF
Interrupt Flag	-IF
Trap Flag	-TF
Zero Flag	-ZF
Auxiliary Flag	-AF
Parity Flag	-PF
Carry Flag	-CF

add.asm

```
org 0x0100

section text:

start:
mov ax,10
mov bx,20
add ax, bx
mov ax,0x4c00        ; exit
int 0x21             ; OS should perform the above operation
```



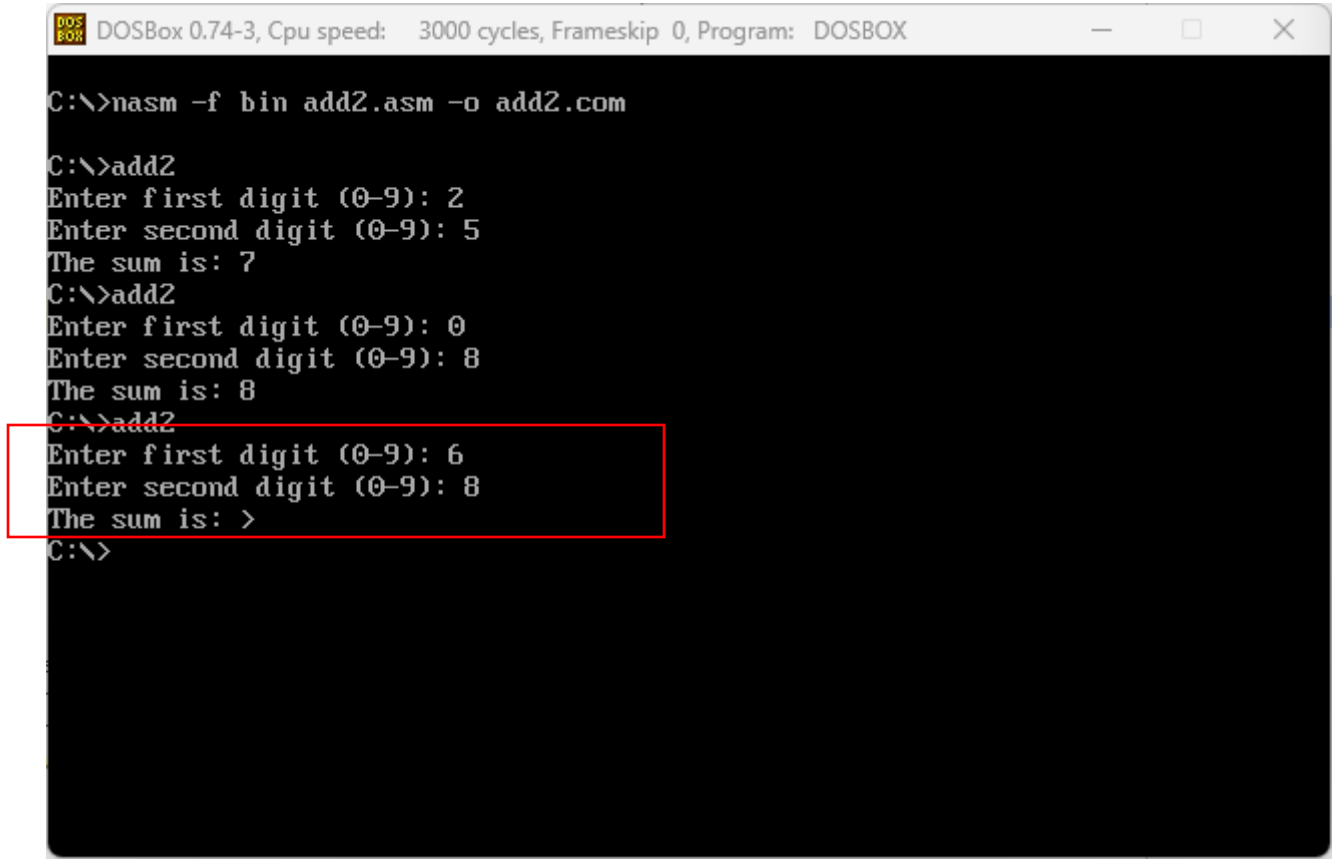
nasm –f bin add.asm –l add.lst –o add.com
Insight add.com

F1: help window

F8: step over

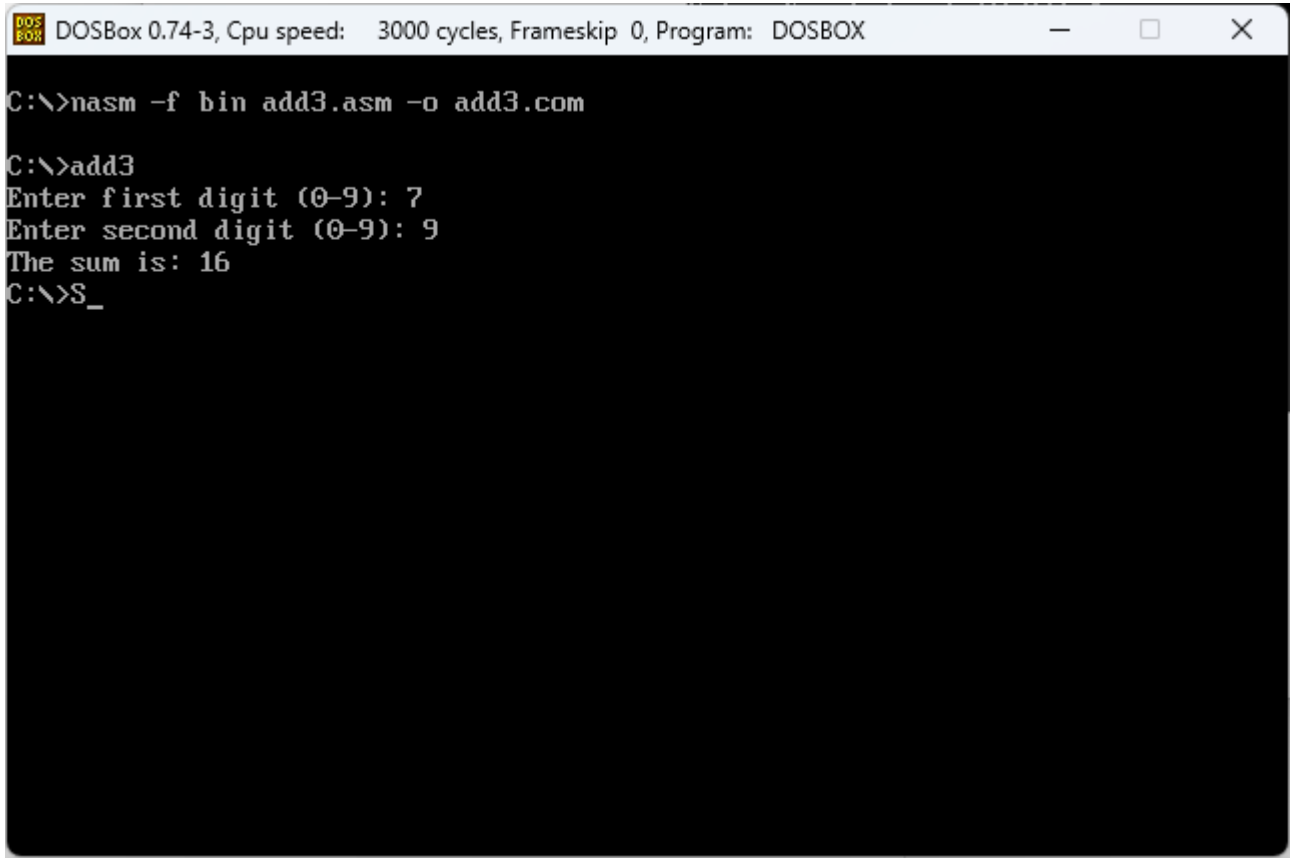
add2.asm

```
org 100h  ; COM file origin

section .text
start:
   ; Prompt and get first digit
   mov ah, 09h         ; DOS service: output string
   lea dx, prompt1     ; Load the address of prompt1
   int 21h             ; Call DOS interrupt

   mov ah, 01h         ; DOS service: input single character
   int 21h             ; Call DOS interrupt
   sub al, '0'         ; Convert ASCII to integer (0-9)
   mov bl, al          ; Store the first number in BL

   ; Prompt and get second digit
   mov ah, 09h         ; DOS service: output string
   lea dx, prompt2     ; Load the address of prompt2
   int 21h             ; Call DOS interrupt

   mov ah, 01h         ; DOS service: input single character
   int 21h             ; Call DOS interrupt
   sub al, '0'         ; Convert ASCII to integer (0-9)
   add bl, al          ; Add the second number to BL (result in BL)

   ; Display "The sum is" prompt
   mov ah, 09h         ; DOS service: output string
   lea dx, sum_prompt  ; Load the address of sum_prompt
   int 21h             ; Call DOS interrupt
```



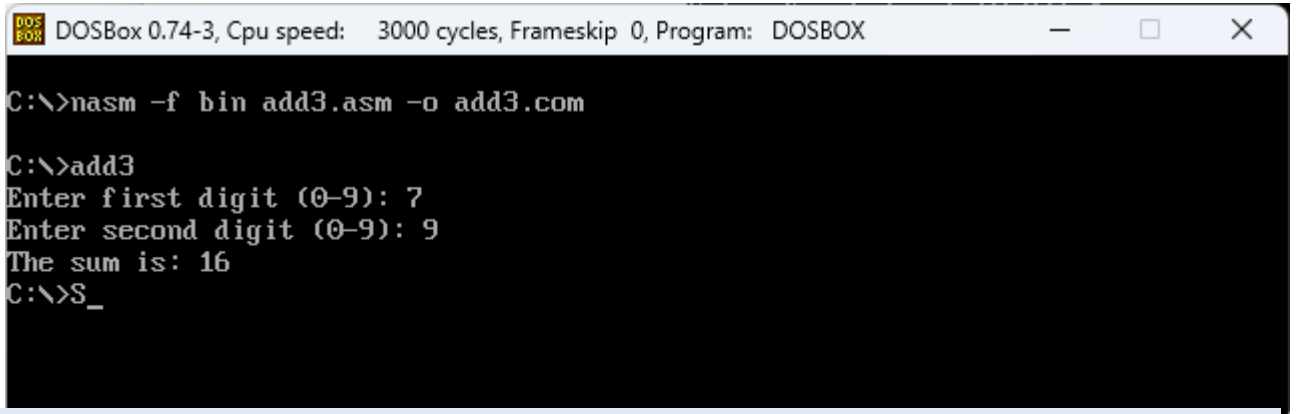
add3.asm

```
org 100h  ; COM file origin

section .text
start:
   ; Prompt and get first digit
   mov ah, 09h         ; DOS service: output string
   lea dx, prompt1      ; Load the address of prompt1
   int 21h             ; Call DOS interrupt

   mov ah, 01h         ; DOS service: input single character
   int 21h             ; Call DOS interrupt
   sub al, '0'         ; Convert ASCII to integer (0-9)
   mov bl, al          ; Store the first number in BL

   ; Prompt and get second digit
   mov ah, 09h         ; DOS service: output string
   lea dx, prompt2      ; Load the address of prompt2
   int 21h             ; Call DOS interrupt

   mov ah, 01h         ; DOS service: input single character
   int 21h             ; Call DOS interrupt
   sub al, '0'         ; Convert ASCII to integer (0-9)
   add bl, al          ; Add the second number to BL (result in BL)

   ; Display "The sum is" prompt
   mov ah, 09h         ; DOS service: output string
   lea dx, sum_prompt    ; Load the address of sum_prompt
   int 21h             ; Call DOS interrupt
```



add3.asm

```
org 100h  ; COM file origin
section .text
start:
    ; Prompt and get first digit
    mov ah, 09h          ; DOS service: output string
    lea dx, prompt1      ; Load the address of prompt1
    int 21h              ; Call DOS interrupt

    mov ah, 01h          ; DOS service: input single character
    int 21h              ; Call DOS interrupt
    sub al, '0'          ; Convert ASCII to integer (0-9)
    mov bl, al           ; Store the first number in BL

    ; Prompt and get second digit
    mov ah, 09h          ; DOS service: output string
    lea dx, prompt2      ; Load the address of prompt2
    int 21h              ; Call DOS interrupt

    mov ah, 01h          ; DOS service: input single character
    int 21h              ; Call DOS interrupt
    sub al, '0'          ; Convert ASCII to integer (0-9)
    add bl, al           ; Add the second number to BL (result in BL)

    ; Display "The sum is" prompt
    mov ah, 09h          ; DOS service: output string
    lea dx, sum_prompt    ; Load the address of sum_prompt
    int 21h              ; Call DOS interrupt

    ; Check if the sum is greater than 9
    cmp bl, 9            ; Compare the sum in BL with 9
    jg output_two_digits  ; If greater than 9, jump to two-digit output

    ; Handle single-digit sum
    add bl, '0'          ; Convert result back to ASCII
    mov ah, 02h          ; DOS service: output character
    mov dl, bl           ; Load the result (in ASCII) into DL
    int 21h              ; Call DOS interrupt
    jmp done             ; Jump to end of program
```




```
output_two_digits:
    ; Handle two-digit sum (greater than 9)
    sub bl, 10           ; Subtract 10 to get the tens place
    mov dl, '1'          ; Tens place is 1 (since sum > 9 and less than 20)
    mov ah, 02h          ; DOS service: output character
    int 21h              ; Call DOS interrupt

    ; Display the ones place (remaining in BL)
    add bl, '0'          ; Convert the ones digit back to ASCII
    mov dl, bl           ; Load DL with ones digit
    mov ah, 02h          ; DOS service: output character
    int 21h              ; Call DOS interrupt

done:
    ; Exit the program
    mov ah, 4Ch          ; DOS service: terminate program
    int 21h              ; Call DOS interrupt

section .data
prompt1 db 'Enter first digit (0-9): $'
prompt2 db 0Dh, 0Ah, 'Enter second digit (0-9): $'
sum_prompt db 0Dh, 0Ah, 'The sum is: $'
```